BFSI Multi-Agent Intelligence System using Agentic RAG, LangGraph, FastAPI and Azure OpenAI

Abstract

The Banking, Financial Services, and Insurance (BFSI) sector generates large volumes of annual reports, regulatory documents, banking reform publications, financial inclusion reports, and policy updates. Extracting meaningful insights from such documents is often time-consuming and requires significant domain expertise. Traditional search systems are limited in their ability to understand context, perform document-level reasoning, and generate structured insights from multiple years of reports.

This project presents a **BFSI Multi-Agent Intelligence System** built using **Agentic Retrieval-Augmented Generation (Agentic RAG)**, **LangGraph**, **Azure OpenAI**, **FAISS**, **BM25 Retrieval**, **FastAPI**, and **Streamlit**. The system utilizes annual reports published by the Department of Financial Services (DFS), Ministry of Finance, Government of India, as its knowledge base. The solution employs a hybrid retrieval mechanism combining semantic search and keyword-based retrieval, enabling highly relevant information discovery across multiple annual reports.

A multi-agent workflow consisting of a Planner Agent, Agentic RAG Agent, BFSI Analyst Agent, Executive Reporter Agent, and Evaluator Agent was implemented using LangGraph. The architecture separates retrieval, analysis, reporting, and validation responsibilities, producing explainable and evidence-based responses grounded in DFS annual reports.

The system demonstrates the practical application of Agentic AI in BFSI intelligence, regulatory analysis, policy understanding, and financial inclusion research.

Chapter 1: Introduction

1.1 Background

The BFSI sector is one of the most information-intensive industries. Financial institutions, policymakers, analysts, regulators, and researchers continuously generate and consume annual reports, policy documents, regulatory circulars, financial statistics, and strategic publications.

The Department of Financial Services (DFS), Ministry of Finance, publishes annual reports covering:

- Financial Inclusion
- Public Sector Banks
- Insurance Sector Developments
- Pension Reforms
- Regulatory Initiatives
- Banking Sector Reforms
- Kisan Credit Card Initiatives
- PMJDY, PMJJBY, PMSBY Schemes
- EASE Reforms
- Financial Stability Measures

Although these reports provide valuable insights, manually analyzing multiple years of documents is difficult, inefficient, and prone to oversight.

1.2 Problem Statement

Department of Financial Services annual reports contain valuable information spread across hundreds of pages and multiple years. Users often require answers to questions such as:

- What reforms were introduced in EASE?
- What is the Kisan Credit Card Scheme?
- How have financial inclusion initiatives evolved?
- What policy measures were adopted in specific years?

Traditional keyword search methods are insufficient because they lack semantic understanding and analytical reasoning capabilities. Large Language Models provide reasoning capabilities but may hallucinate when not grounded in enterprise knowledge.

The challenge is to create a system capable of:

- Retrieving relevant information efficiently.
- Understanding BFSI-specific terminology.
- Generating grounded responses.
- Producing analytical insights.
- Supporting multi-year reasoning.
- Validating the quality of generated outputs.

1.3 Project Objectives

The primary objectives of the project are:

1. Build a BFSI-focused knowledge retrieval system.
2. Implement Agentic RAG using LangGraph.
3. Create a multi-agent workflow for BFSI intelligence.
4. Integrate Azure OpenAI for embeddings and reasoning.
5. Implement Hybrid Retrieval using FAISS and BM25.
6. Develop a FastAPI backend.
7. Build an interactive Streamlit interface.
8. Implement CI/CD using GitHub Actions.
9. Generate grounded and explainable responses.

Chapter 2: Dataset Description

2.1 Dataset Source

The project utilizes annual reports released by the Department of Financial Services (DFS), Ministry of Finance.

Reports Used

Annual Report 2020-21
Annual Report 2021-22
Annual Report 2022-23
Annual Report 2023-24
Annual Report 2024-25
Annual Report 2025-26

2.2 Information Available

The reports contain information related to:

- Banking Reforms
- Financial Inclusion
- Kisan Credit Card (KCC)
- PMJDY
- PMJJBY
- PMSBY
- Public Sector Banks
- Insurance Companies
- Pension Reforms
- EASE Reforms
- RRB Reforms
- Digital Banking
- Regulatory Initiatives

2.3 Dataset Characteristics

Attribute	Description
Format	PDF
Domain	BFSI
Language	English
Source	Department of Financial Services

Years Covered	2020-2026
Document Type	Annual Reports

Chapter 3: Technology Stack

Programming Language

- Python 3.12

AI Services

- Azure OpenAI
- Azure OpenAI Embeddings
- Azure GPT Deployment

Frameworks

- LangChain
- LangGraph

Retrieval Technologies

- FAISS
- BM25 Retriever
- Ensemble Retriever

Backend

- FastAPI

Frontend

- Streamlit

Supporting Libraries

langchain
langchain-community
langchain-core
langchain-openai
faiss-cpu
langchain-cohere
rank_bm25
python-dotenv
fastapi
streamlit
sentence-transformers
requests
grandalf

Chapter 4: System Architecture

High-Level Architecture

DFS Annual Reports
|
▼
Document Loading
|
▼
Text Chunking
|
▼
Azure OpenAI Embeddings
|
▼
FAISS Vector Database
|
▼
Hybrid Retrieval
(FAISS + BM25)
|
▼
LangGraph Workflow
|
▼
FastAPI Backend
|
▼
Streamlit Frontend

Chapter 5: Knowledge Base Construction

Document Loading

All PDF reports are loaded using:

Python
DirectoryLoader
PyPDFLoader

This enables automated ingestion of multiple DFS annual reports.

Text Chunking

The reports are divided into smaller segments using:

Python
RecursiveCharacterTextSplitter

Configuration:

Chunk Size = 500
Chunk Overlap = 100

Benefits:

- Improved retrieval accuracy
- Better context preservation
- Reduced information loss

Embedding Generation

The chunked documents are transformed into dense vector embeddings using:

Python

AzureOpenAIEmbeddings

These embeddings capture semantic meaning rather than exact textual matches.

FAISS Index Creation



Embeddings are stored in:

Python
FAISS

to support efficient vector similarity search.

Generated files:

index.faiss
index.pkl

The index serves as the primary semantic knowledge base used by the retrieval system.

Chapter 6: Hybrid Retrieval-Augmented Generation

Hybrid Retrieval Strategy

The project utilizes Hybrid Search by combining:

Semantic Search (FAISS)
+
Keyword Search (BM25)

Semantic Search

Implemented using:

Python
FAISS Retriever

Purpose:

- Capture semantic meaning
- Handle synonyms
- Retrieve conceptually related information

Maximum Marginal Relevance (MMR)

The semantic retriever uses:

Python
search_type="mmr"

Purpose:

Relevance
+
Diversity
``

Instead of returning repetitive chunks, MMR provides a diverse set of highly relevant chunks.

BM25 Retriever

Implemented using:

Python
BM25Retriever

Purpose:

- Exact keyword matching
- Acronym retrieval
- Policy and regulation retrieval

Ensemble Retriever

Both retrieval approaches are combined using:

Python
EnsembleRetriever

Configuration:

FAISS Weight = 0.6
BM25 Weight = 0.4

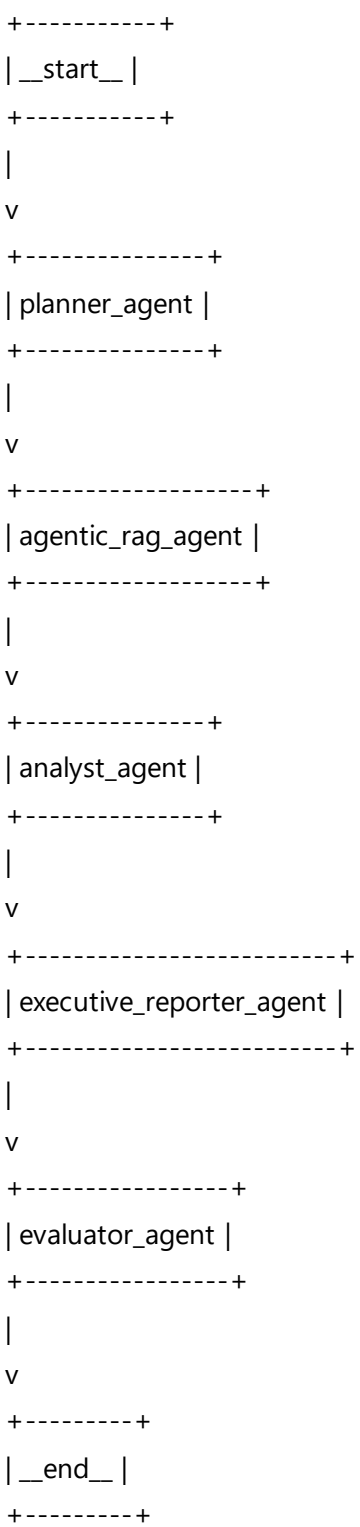
Advantages:

- Improved recall
- Improved precision
- Better context quality
- Reduced retrieval failure

Chapter 7: LangGraph Multi-Agent Workflow

The project employs LangGraph to implement a multi-agent architecture.

Workflow Diagram



Planner Agent

Purpose

The Planner Agent analyzes user intent and creates a retrieval strategy.

Responsibilities

- Query classification
- Topic identification
- Requirement extraction
- Retrieval planning

Example Output:

Query Type
Topic
Required Information
Relevant Years

Agentic RAG Agent

Purpose

Responsible for knowledge retrieval.

Tool Used

Hybrid BFSI RAG Tool

Retrieval Pipeline

User Question
↓
FAISS Search

+
BM25 Search
↓
Ensemble Retrieval
↓
Relevant Chunks

Output:

Retrieved Context
Retrieved Documents
Source References

BFSI Analyst Agent

Purpose

Analyze retrieved evidence.

Responsibilities

Generate:

- Key Findings
- Policy Insights
- Regulatory Observations
- Trend Analysis
- Strategic Implications

This agent acts as the primary reasoning component.

Executive Reporter Agent

Purpose

Transform analytical findings into business-ready reports.

Output Structure

Executive Summary

Key Findings

Trend Analysis

Strategic Implications

Conclusion

Evaluator Agent

Purpose

Quality Assurance and Validation.

Evaluation Criteria

Checks:

Query Alignment
Evidence Grounding
Completeness
Analytical Quality
Hallucination Risk

Produces:

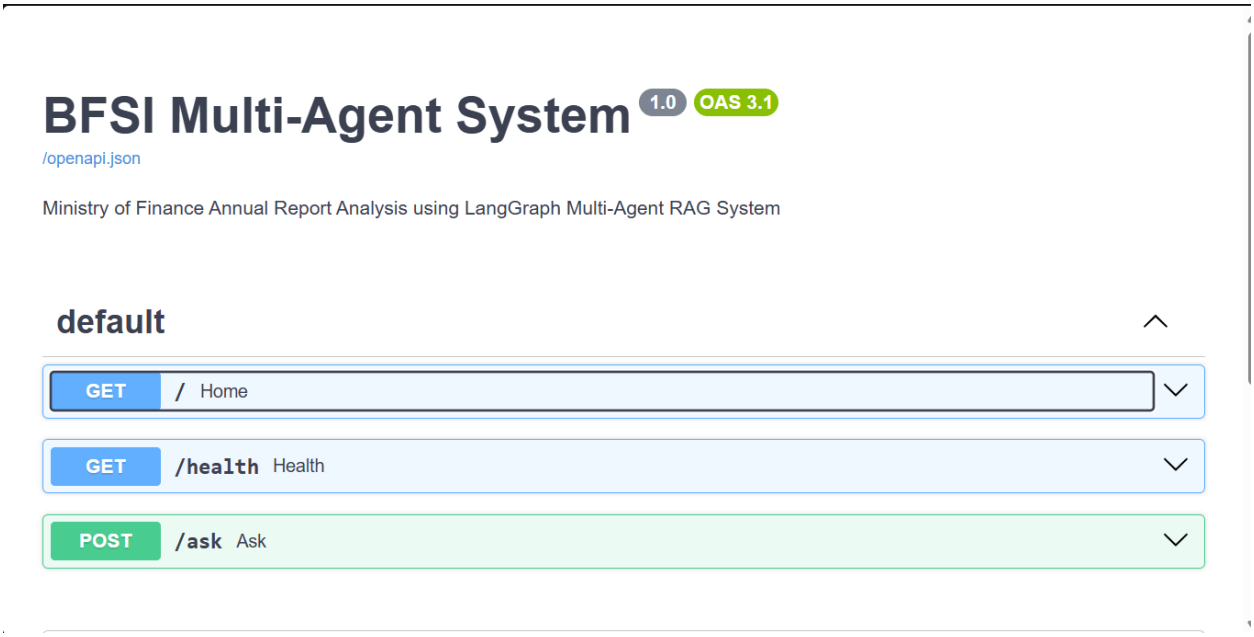
APPROVED

or

NEEDS_IMPROVEMENT

Chapter 8: Backend Implementation

FastAPI Backend



The backend was developed using FastAPI.

Endpoints

Root Endpoint

HTTP
GET /

Purpose:

Service validation.

Health Endpoint

HTTP
GET /health

Purpose:

Application monitoring.

Ask Endpoint

HTTP
POST /ask

Purpose:

Execute complete LangGraph workflow.

Input:

JSON
{
"question":"What is Kisan Credit Card?"

}

Output:

JSON
{
"answer":"..."
}

Chapter 9: Frontend Implementation

Streamlit Application



The frontend was implemented using Streamlit.

Features

- Query Input
- Chat-style Interface
- BFSI Question Answering
- FastAPI Integration
- Session-Based Chat History

User Interaction Flow

User Query
↓
Streamlit UI
↓
FastAPI API
↓
LangGraph Workflow
↓
Response
↓
UI Display

Chapter 11: Testing and Results

Test Case 1: Kisan Credit Card

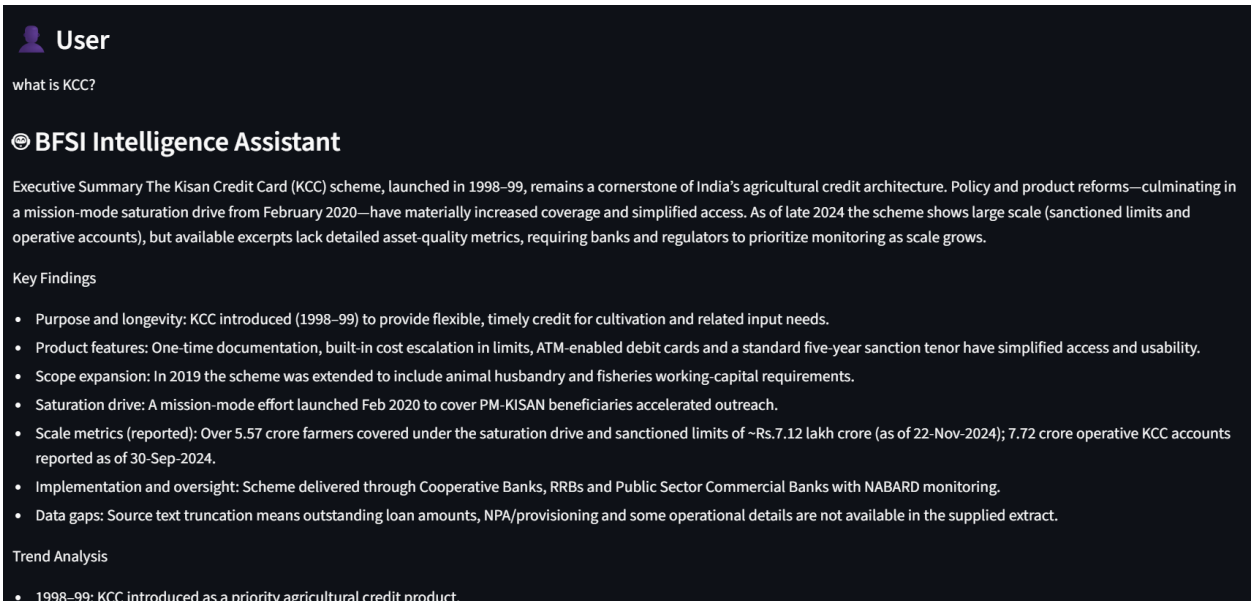
Query

What is Kisan Credit Card?

Result

The system successfully identified:

- Purpose
- Credit limits
- Interest Subvention
- PM-KISAN Integration
- Debit Card Functionality



Test Case 2: EASE Reforms

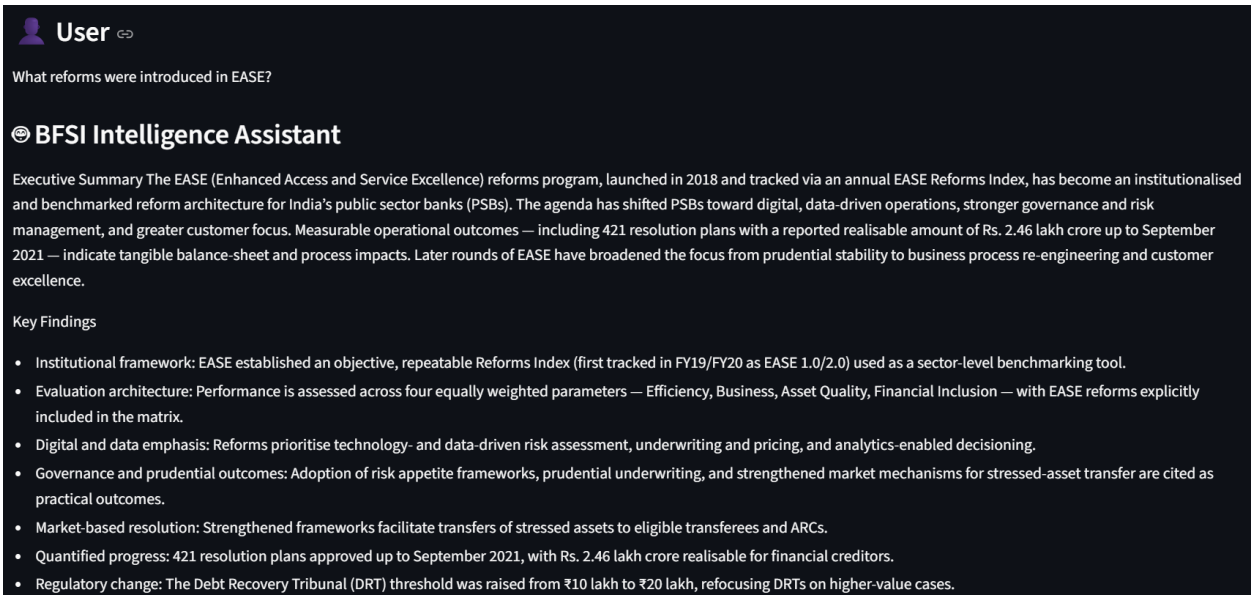
Query

What reforms were introduced in EASE?

Result

The system retrieved and summarized:

- Governance Reforms
- Risk Appetite Frameworks
- Loan Management Systems
- Early Warning Signals
- Digital Banking Initiatives



Chapter 12: Challenges and Solutions

Challenge 1: Retrieval Quality

Problem

Keyword search missed semantic concepts.

Solution

Hybrid Retrieval:

FAISS + BM25

Challenge 2: Hallucination Prevention

Problem

Large language models may generate unsupported answers.

Solution

- Grounded RAG Prompt
- Evaluator Agent
- Source Traceability

Challenge 3: Multi-Year Analysis

Problem

Information distributed across multiple reports.

Solution

Analyst Agent capable of aggregating evidence from multiple years.

Challenge 4: BFSI Domain Complexity

Problem

Regulatory terminology and policies require domain understanding.

Solution

Domain-specific prompts and BFSI-focused retrieval.

Chapter 13: Future Enhancements

Planned improvements include:

- RAGAS Evaluation
- BLEU and ROUGE Metrics
- Relevance Validation Layer
- Query Rewriting

- Cohere Reranking Integration
- Cloud Deployment
- Dockerization
- Human-in-the-Loop Review
- Pinecone or Azure AI Search Migration

Chapter 14: Conclusion

This project successfully demonstrates an Agentic RAG-based BFSI Intelligence System using Azure OpenAI, LangGraph, FastAPI, and Streamlit.

The system combines:

- Hybrid Retrieval
- Multi-Agent Reasoning
- Evidence-Based Responses
- Regulatory Intelligence
- Explainable Outputs

to create a scalable BFSI knowledge assistant capable of answering regulatory, financial inclusion, and banking-related questions grounded in Department of Financial Services annual reports.

The project highlights how Agentic AI can be effectively used to transform large collections of enterprise documents into an intelligent decision-support system for financial services and regulatory analysis.